



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Documento de Estándares de Programación

Sistema NexusLib

Curso: Patrones de Software

Docente: Ing. Patrick Cuadros Quiroga

Integrantes:

<i>Hurtado Ortiz, Leandro Diego</i>	<i>(2015052384)</i>
<i>Flores Navarro, Eduardo Gino</i>	<i>(2023076793)</i>
<i>Cortez Mamani, Julio Samuel</i>	<i>(2023077283)</i>

**Tacna – Perú
2026**

Sistema NexusLib	Versión: 1.0
Documento de Estándares de Programación	

Sistema NexusLib
Documento de Estándares de Programación
Versión 1.0

Sistema NexusLib	Versión 1.0
Documento de Estándares de Programación	

Historia de Revisión

Historial de revisiones				
Ítem	Fecha	Versión	Descripción	Equipo
1	10/06/2026	1.0	Versión Final.	

Tabla de Contenidos

1. OBJETIVO	4
2. DECLARACIÓN DE VARIABLES	5
2.1. Descripción de la Variable.	5
2.2. Variables de Tipo Arreglo	6
3. Definición de Controles	7
3.1. Prefijo para el Control	7
3.2. Declaración de variables, atributos y objetos	7
3.3. Declaración de clases	8
3.4. Declaración de métodos	8
3.5. Control de versiones de código fuente	9
4. Clases	10
5. Métodos, Procedimientos y Funciones definidos por el Usuario.	10
6. Beneficios	11
7. Conclusiones	13

Sistema NexusLib	Versión: 1.0
Documento de Estándares de Programación	

Estándares de Programación

1. OBJETIVO

Reglamentar la forma en que se implementará el código fuente del proyecto, pasando, por las variables, controles, clases, métodos, ficheros, archivos y todo aquello que esté implicado en el código,

Mejorar y uniformizar a través de las reglas que se proponen, el estilo de programación que tiene cada programador.

- Los nombres de variables serán mnemotécnicos con lo que se podrá saber el tipo de dato de cada variable con sólo ver el nombre de la variable.
- Los nombres de variables serán sugestivos, de tal forma que se podrá saber el uso y finalidad de dicha variable o función fácilmente con solo ver el nombre de la variable.
- La decisión de poner un nombre a una variable o función será mecánica y automática, puesto que seguirá las reglas definidas por nuestro estándar.
- Permite el uso de herramientas automáticas de verificación de nomenclaturas.

Por tanto, se seguirán dichos patrones para un entendimiento legible del código y para facilitar el mantenimiento del mismo.

2. DECLARACIÓN DE VARIABLES

Se propone que la declaración de las variables se ajuste rigurosamente al motivo para el cual sean requeridas dentro del flujo del sistema. El mnemotécnico definido se establece tomando en consideración principalmente los siguientes lineamientos institucionales:

- **Longitud Sugestiva:** El identificador debe ser lo suficientemente claro para describir su propósito sin saturar el código. Para el entorno de desarrollo web de NexusLib, se establece una longitud máxima por variable de **16 caracteres**.
- **Formato de Escritura:** Se empleará la nomenclatura **CamelCase** (la primera palabra inicia en minúscula y las iniciales de las palabras subsecuentes se escriben en Mayúsculas).
- **Sintaxis en el Entorno Web:** En la capa lógica de los microservicios, toda variable irá precedida por el signo de dólar (\$) conforme a la sintaxis nativa de PHP, mientras que en los scripts de comportamiento del frontend se inicializarán bajo el estándar moderno de JavaScript.

Alcance	Prefijo	Ejemplo
Nivel de la clase	<i>Ninguno</i>	SearchQuery

- El tipo de dato al que pertenece la variable.

Por lo tanto la estructura de la variable es como sigue:

Estructura	Descripción de la Variable
LONGITUD. MAX.	1 16
FORMATO	Mayúscula la primera parte y luego la segunda con Mayúsculas
EJEMPLO	SearchQuery

Siendo el nombre que identifica a la variable: **searchQuery**

2.1. Descripción de la Variable.

Nombre que se le asignará a la variable para que se le identifique y deberá de estar asociada al motivo para la cual se le declara.

Ejemplos: uuid, searchQuery, authKey, registroId, isVerified

Sistema NexusLib	Versión: 1.0
Documento de Estándares de Programación	

2.2. Variables de Tipo Arreglo

En el caso de las definiciones de arreglos de elementos se declarará la variable en plural, el cual nos dará a entender que se trata de una variable del tipo arreglo la cual contendrá de cero a más datos, según el tamaño declarado.

Ejemplos (Estructuras de Backend PHP):

```
$books = []; // Matriz general de registros bibliográficos
$favorites = []; // Colección de favoritos del usuario
$reservations = []; // Lista de apartados físicos activos
$alphaResults = []; // Libros digitales extraídos desde Alpha Cloud
$elibroResults = []; // Metadatos obtenidos desde e-Libro
```

Ejemplos (Estructuras de Frontend JavaScript):

```
const activeFilters = []; // Criterios de filtrado seleccionados por el alumno
const displayedBooks = []; // Elementos bibliográficos impresos en pantalla
```

3. Definición de Controles

Se propone que la declaración de las variables asociadas a los controles de la interfaz de usuario web (DOM) y los componentes lógicos de backend se ajusten a las siguientes directrices mnemotécnicas:

3.1. Prefijo para el Control

El prefijo del control será determinado mediante tres caracteres que estarán conformados por las consonantes más representativas del control. Al tratarse de un entorno web nativo, estos prefijos identifican la naturaleza de los elementos en el árbol del DOM:

- El control **Button** (<button>), estará asociado al prefijo **btn**.
- El control de entrada de texto **Input Text** (<input type="text">), estará asociado al prefijo **txt**.
- El control de selección desplegable **Select** (<select>), estará asociado al prefijo **lst**.
- El contenedor dinámico estructurado **Div** (<div>), estará asociado al prefijo **div**.

3.2. Declaración de variables, atributos y objetos

Se debe declarar una variable por línea.

Título	Descripción
Sintaxis	• <code>[\$[Nombre de la Variable]</code> (Para backend en PHP) • <code>let [Nombre de la Variable] / const [Nombre de la Variable]</code> (Para frontend en JS)
Descripción	Todas las variables o atributos tendrán una longitud máxima de 30 caracteres. El nombre de la variable puede incluir más de un sustantivo los cuales se escribirán juntos en formato camelCase.
Observaciones	En la declaración de variables o atributos no se deberá utilizar caracteres como: • Letra Ñ o ñ. • Caracteres especiales <code>¡, ^, #, \$, %, &, /, (,), ¿, ', +, -, *, {, }, [,]</code>. • Caracteres tildados: á, é, í, ó, ú.
Ejemplo	<code>public string \$email;</code> Indica un atributo nativo de clase en el backend para correos. <code>let searchQuery;</code> Indica una variable local en JavaScript para capturar la barra de búsqueda.

3.3. Declaración de clases

Título	Descripción
Sintaxis	<code>class [Nombre de Clase]</code>
Descripción	El nombre de las clases tendrá una longitud máxima de 30 caracteres y las primeras letras de todas las palabras estarán en mayúsculas (Formato PascalCase). Al trabajar con programación orientada a objetos en PHP moderno, no se emplean prefijos obsoletos (como <code>cls</code>).
Observaciones	En la declaración de clases no se deberá utilizar caracteres como: • Letra Ñ o ñ. • Caracteres especiales <code>¡, ^, #, \$, %, &, /, (,), ¿, ', +, -, *, {, }, [,]</code>. • Caracteres tildados: <code>á, é, í, ó, ú</code>.
Ejemplo	<pre>class GatewayController</pre> Indica la clase controladora del API Gateway en el orquestador. <pre>class InventoryRepository</pre> Indica la clase encargada del acceso a datos del inventario.

3.4. Declaración de métodos

Título	Descripción
Sintaxis	<code>[visibilidad] function nombreMetodo([listaParámetros])</code>
Descripción	El nombre del método constará hasta de 25 caracteres. La primera letra de la primera palabra del nombre será escrita en minúscula y las siguientes palabras empezarán con letra mayúscula (Nomenclatura camelCase).
Observaciones	En la declaración de métodos no se deberá utilizar caracteres como: • Letra Ñ o ñ. • Caracteres especiales <code>¡, ^, #, \$, %, &, /, (,), ¿, ', +, -, *, {, }, [,], _</code>. • Caracteres tildados: <code>á, é, í, ó, ú</code>.
Ejemplo	<pre>private function aggregateSearchFields(array \$services)</pre> Indica un método interno para unificar los campos de los catálogos en el Gateway.

Reglas de nomenclatura

- Usar nombres autoexplicativos: el método debe permitir entender su propósito sin necesidad de inspeccionar su contenido.
- Iniciar con minúscula (camelCase).
- No exceder los 25 caracteres (recomendado, no obligatorio si afecta la claridad).
- No usar guiones bajos ni espacios en blanco.
- Utilizar inglés como idioma base del nombre, salvo que el proyecto especifique lo contrario.

Sistema NexusLib	Versión 1.0
Documento de Estándares de Programación	

Ejemplos

```
private function forwardRequest()
public function generateToken(string $userId)
protected function validateEmail(string $email)
```

- Evitar abreviaturas innecesarias.
- Usar verbos estándar como obtener (*get*), validar (*validate*), calcular (*calculate*), enviar (*send*), procesar (*process*), actualizar (*update*).
- Mantener consistencia entre todos los métodos del proyecto.

Declaración de funciones

Título	Descripción
Sintaxis	function nombreFuncion([listaParámetros]): [TipoDato]
Descripción	El nombre del objeto constará hasta de 25 caracteres, no es necesario colocar un nombre que indique la clase a la cual pertenece. La primera letra de la primera palabra del nombre será escrita en mayúsculas. El tipo de dato de retorno se coloca al final y será obligatorio colocarlo.
Observaciones	En la declaración de objetos no se deberá utilizar caracteres como: • Letra Ñ o ñ. • Caracteres especiales ¡, ^, #, \$, %, &, /, (,), ¿, ', +, -, *, {, }, [,], _. • Caracteres tildados: á, é, í, ó, ú.
Ejemplo	public function SendEmail(string \$recipient, string \$subject, string \$htmlBody): void Indica una función para enviar un correo electrónico.

3.5. Control de versiones de código fuente

Cada modificación realizada será guardada de la forma:

Título	Descripción
Formato	[NOMBRE DOCUMENTO][_][FECHA][_][HORA] donde y la fecha estará en formato yyymmdd y la hora en formato HHMM.
Descripción	Se generarán archivos con las siguientes extensiones: .zip o .rar.

Sistema NexusLib	Versión: 1.0
Documento de Estándares de Programación	

4. Clases

El nombre de las clases debe ser auto descriptivo de manera que no se requiera, en lo posible, entrar al código de la función para saber qué es lo que realiza.

- Ejemplos: GatewayController, AuthController, InventoryRepository, SavedBooksService

Nota:

- No se hará uso de los caracteres: Espacio en blanco " ", Caracter de subrayado "_".

5. Métodos, Procedimientos y Funciones definidos por el Usuario.

El nombre de las funciones y procedimientos debe ser auto descriptivo de manera que no se requiera, en lo posible, entrar al código de la función para saber qué es lo que realiza. **verbo-Sustantivo**

El estándar para nombres de procedimiento es usar un Verbo que describa la acción realizada seguida por un sustantivo (objeto sobre el cual actúa). Se recomienda:

- Usar un nombre que represente una acción y un objeto.
- El nombre del procedimiento debe indicar qué hace el procedimiento a... o qué hace el procedimiento con...
- El verbo debe estar en infinitivo.
- Ser consistente en el orden de las palabras. Si se va a usar *verboNombre*, siempre usar *verboNombre*.
- Ser consistente en los verbos y sustantivos usados. Por ejemplo, si tiene un procedimiento *asignarNombre*, en vez de *colocarNombre*.

Para la acción *modificar cuentas del cliente* se define: **modificarCuenta**

- **Verbo:** modificar
- **Sustantivo:** Cuenta

Ejemplos aplicados en NexusLib:

- Para la acción *verificar tokens de sesión*: verifyToken
- Para la acción *agregar libros a favoritos*: saveBook
- Para la acción *consultar disponibilidad de stock*: checkStock
- Para la acción *sincronizar estados lógicos*: syncState

Nota:

- No se hará uso de los caracteres: Espacio en blanco " ", Caracter de subrayado "_".
- La nomenclatura de argumentos o parámetros pasados a los procedimientos/funciones así como para valores devueltos por funciones sigue las mismas convenciones que la nomenclatura para variables.

Sistema NexusLib	Versión 1.0
Documento de Estándares de Programación	

6. Beneficios

Mayor productividad

- Al evitar diferencias innecesarias en estilo, el equipo reduce el tiempo dedicado a interpretar código ajeno y se enfoca en desarrollar nuevas funcionalidades distribuidas dentro del ecosistema web.

Facilita el mantenimiento

- Un código consistente y legible permite detectar y corregir errores más rápidamente, lo que reduce el costo de mantenimiento a largo plazo en componentes clave como el API Gateway o los servicios core.

Mejor colaboración en equipo

- Todos los integrantes trabajan con las mismas convenciones, lo que mejora la integración de código, la revisión entre pares y la incorporación de nuevos desarrolladores al proyecto.

Reduce errores y ambigüedades

- Al tener reglas claras de nomenclatura, indentación y estructura, se minimizan los malentendidos y errores comunes durante el intercambio asíncrono de JSONs en el backend.

Escalabilidad del proyecto

- Proyectos grandes y de largo plazo se benefician de una base de código uniforme que soporta el crecimiento continuo y la adición de nuevos conectores de catálogos sin volverse caótico.

Documentación implícita

- Un código bien estructurado según estándares sirve como documentación viva, facilitando el entendimiento sin necesidad de comentarios excesivos en las clases lógicas.

Sistema NexusLib	Versión: 1.0
Documento de Estándares de Programación	

Control de versiones

Se utilizará **Git** como sistema de control de versiones distribuido, debido a su eficiencia, flexibilidad y amplia adopción en la industria del desarrollo de software.

Estrategia de ramificación

Se empleará el modelo de ramificación basado en funcionalidades (**Feature Branching**), siguiendo estas recomendaciones:

1. Rama principal (master)

- Siempre debe contener código estable y listo para producción en la máquina virtual de Azure.
- Solo se fusionan cambios revisados y validados.

2. Ramas de desarrollo ([nombre_de_usuario])

- Contiene la última versión y espacio de trabajo de cada desarrollador del equipo.
- Se integran las funcionalidades del catálogo unificado antes de ser llevadas a producción.

3. Ramas de características (feature/nombre-funcionalidad)

- Se crean a partir del entorno base.
- Cada rama contiene una nueva funcionalidad o mejora específica (ej. feature/scrapper-alpha, feature/favorites-logic).
- Al finalizar, se realiza una solicitud de *pull request* para revisión y fusión.

4. Ramas de corrección (hotfix/nombre-error)

- Se crean desde la rama principal cuando es necesario corregir errores urgentes detectados directamente en producción.
- Después de su fusión y despliegue, también se actualizan las ramas locales de desarrollo.

5. Ramas de lanzamiento (release/version-x.y)

- Se utilizan para preparar una nueva versión oficial del sistema.
- Permiten realizar pruebas y ajustes finales de integración sin afectar el flujo continuo de desarrollo.

Sistema NexusLib	Versión 1.0
Documento de Estándares de Programación	

7. Conclusiones

- El documento establece reglas claras sobre cómo declarar variables, clases, métodos y componentes del DOM web, promoviendo un estilo de codificación uniforme en todo el ecosistema de **NexusLib**. Esto facilita significativamente la lectura, la trazabilidad de las peticiones asíncronas y el mantenimiento del código en el backend, independientemente de qué miembro del equipo de ingeniería lo haya desarrollado.
- Se enfatiza el uso de nombres mnemotécnicos y descriptivos bajo formatos normados como *CamelCase* y *PascalCase*, lo que permite identificar rápidamente el propósito de cada variable, consulta o entidad dentro de los microservicios sin necesidad de añadir comentarios excesivos. Esta práctica mejora la autocomprensión de la arquitectura, optimiza la depuración de flujos lógicos distribuidos y reduce drásticamente la necesidad de exploraciones profundas en los archivos fuentes de PHP y JavaScript.
- Al establecer estándares desde el inicio del ciclo de desarrollo, se garantiza que las diferentes capas del sistema (interfaz web cliente, orquestador API Gateway y servicios core de base de datos) interactúen de manera coherente y predecible. Esto no solo eleva los niveles de colaboración técnica y control de versiones mediante Git, sino que también prepara una base sólida y estandarizada para escalar la plataforma hacia nuevos repositorios y catálogos universitarios de forma ordenada y sostenible.